

Mini Project Report

Console Student Record Management Application

Ryane Bose MCA

1. Project Overview

The Student Record Management Application is a console based Python application designed to manage student records. The application allows the user to add new student records, view existing records, search for students, update information, and delete records.

The project was developed as a practical application of the Python concepts covered in the assignment. These include data types and variables, conditional statements, loops, functions, exception handling, file input and output, and menu driven console application design.

Each student record contains four pieces of information: Student ID, Student Name, Student Age, and Student Course. The records are stored permanently in a JSON file so that the information remains available after the program is closed.

2. Project Approach and Structure

The application was divided into three separate files so that different parts of the project have clear responsibilities. This makes the code easier to understand, maintain, and test.

main.py

The main.py file controls the overall flow of the application. It displays the main menu, accepts the user's choice, loads the student data, and calls the appropriate function from the student operations file. It also checks whether the required student data file is available before allowing the application to continue.

student_operations.py

The student_operations.py file contains the main operations performed on student records. It contains functions for searching, adding, viewing, updating, and deleting students. It also contains the functions responsible for loading and saving student data.

students.json

The students.json file is the data file used to store the student records. The records are stored as JSON data so they can be saved to the computer and loaded again when the application starts.

How the files work together

When the application starts, main.py calls the load_student function from student_operations.py. The function reads the records from students.json and returns them to main.py. The records are then passed to the required operation when the user selects an option from the menu.

When a record is added, updated, or deleted, the appropriate function changes the records in memory and calls save_student to write the updated records back to students.json. This allows changes to remain available after the program is closed.

Why JSON was used

JSON was chosen because the student records are represented naturally using Python dictionaries. Each student is stored as a dictionary containing the Student ID, Name, Age, and Course. A collection of these dictionaries can be stored as a list.

Python's json module can directly convert these lists and dictionaries into JSON data and then convert the JSON data back into Python lists and dictionaries. This makes JSON simple and convenient for this project and avoids the need for a more complicated database system.

3. Functions Used in the Project

Functions in main.py

main() Controls the main program. It displays the main menu, loads the student data, checks whether the data file is available, accepts the user's menu choice, and calls the appropriate student operation.

Functions in student_operations.py

search_student(students, field, value) Searches the student records according to the selected field and value. It can search by Student ID or Student Name and returns all matching records.

add_student(students) Collects the Student ID, Name, Age, and Course from the user. It displays the entered information for confirmation and saves the new record when the user confirms the addition.

find_student(students) Provides the view and search menu. It allows the user to find students by ID, Name, Age, or Course, or display all student records.

update_student(students) Finds an existing student using the Student ID or Name and allows the user to change the student's Name, Age, or Course. The change is confirmed before it is saved.

delete_student(students) Finds a student using the Student ID or Name and removes the selected record. The updated list is then saved to the JSON file.

load_student() Reads the students.json file and converts its contents into Python data. The function also handles the situation where the required file does not exist.

save_student(students) Writes the current list of student dictionaries into students.json using the JSON format. This keeps changes stored permanently.

4. Exception Handling

Exception handling was included to prevent common user input errors from causing the application to stop unexpectedly. It also helps the application deal with problems related to the student data file.

Invalid numeric input

A `ValueError` exception is handled whenever the application expects the user to enter a number. This is used for Student ID, Student Age, menu selections, and selections from lists of matching students.

For example, if the user enters text instead of a number, converting the input with `int` would normally cause a `ValueError`. The application catches this error and displays a clear message asking the user to enter a valid number. The user can then try again without the application closing.

Invalid menu choices

The application checks whether menu choices are within the expected range. If the user enters a number outside the available options, the application displays an appropriate message and asks for another choice.

Missing student data file

The `load_student` function handles `FileNotFoundError`. If `students.json` is not present, the function returns `None`. The main function checks this result and stops the application before any student operation can be performed.

This was intentionally implemented so that the application does not create a new data file or allow records to be added when the required student data file is missing.

5. Data Storage

Student records are stored in `students.json`. The Python program uses a list containing dictionaries. Each dictionary represents one student.

The JSON file is stored relative to the location of `student_operations.py`. The program uses Python's `pathlib` module to create the data file path. This ensures that the application looks for `students.json` in the intended project directory instead of depending on the directory from which the program was run.

6. User Interaction

The application uses a menu driven interface. The user can select Add Student, View Students, Update Student, Delete Student, or Quit. During operations, the application provides prompts and status messages so the user knows what the program is currently doing.

Confirmation messages are also used before important changes. This reduces the chance of accidentally adding or changing a student record.

7. Concepts Demonstrated

- Data types and variables are used to store IDs, names, ages, courses, choices, and records.
- Lists and dictionaries are used to organize student information.
- Conditional statements are used to make decisions based on user input and search results.
- Loops are used for the main menu, repeated input, and processing student records.
- Functions divide the application into smaller and manageable parts.
- Exception handling prevents invalid input and missing file errors from unexpectedly stopping the application.
- File input and output are used to read and write student records.
- JSON provides a simple persistent format for storing the Python data structures.
- The menu driven design provides a simple way for users to interact with the application.

8. Conclusion

The Student Record Management Application combines the required Python concepts into one working console application. The project uses separate files to organize the program, Python lists and dictionaries to manage records, JSON for persistent storage, functions for reusable operations, and exception handling for common input and file related problems.

The resulting application provides the basic operations required for managing student records while keeping the implementation straightforward and suitable for a Python programming assignment.